

Módulo 8 - RF 8.1 y RF 8.2

Integrantes: Goya Bautista – Invaldi Juan Martin

1. Alcance del trabajo

El Módulo 8 corresponde a Notificaciones, Documentos y Soporte. Dentro de este módulo existen distintos requerimientos funcionales que serán desarrollados por diferentes grupos.

Nuestro grupo trabajará específicamente sobre:

- **Invaldi Juan Martin, RF 8.1: Notificaciones de viaje.**

Se deberán contemplar notificaciones relacionadas con la solicitud, asignación, arribo, inicio, cancelación y finalización de un viaje.

- **Goya Bautista, RF 8.2: QR de verificación.**

Se deberá generar un QR temporal, asociado a un viaje, de un solo uso y sin exponer información sensible.

Estos dos requerimientos forman parte de una única API del Módulo 8.

Lo que significa que Notificaciones y QR no serán aplicaciones independientes, sino funcionalidades diferentes dentro del mismo servicio. Más adelante, los requerimientos desarrollados por los demás grupos del Módulo 8 también deberán incorporarse a esta misma API.

2. API REST con Node.js y Express

REST es el estilo arquitectónico utilizado para organizar la comunicación entre aplicaciones mediante HTTP. En nuestro caso permitirá representar las funcionalidades de Notificaciones y QR mediante recursos y endpoints definidos.

Express será utilizado dentro de Node.js para implementar el servidor y construir esos endpoints.

Por lo tanto, REST define cómo organizamos la API, mientras que Express es la tecnología que utilizamos para programarla.

Inicialmente, la API de nuestro grupo contemplará recursos relacionados con Notificaciones y QR.

Por ejemplo:

Notificaciones

GET /notifications

POST /notifications

QR

POST /qr

POST /qr/validate

Estos endpoints son una propuesta inicial y podrán ajustarse al momento de definir formalmente el contrato de la API.

3. Una sola API para el Módulo 8

El Módulo 8 se implementará como un único servicio, el cual expondrá una API REST con los endpoints correspondientes a sus distintos requerimientos funcionales.

Dentro de esa API existirán distintas áreas funcionales:

Notifications: correspondiente a RF-8.1.

QR: correspondiente a RF-8.2.

Documents: correspondiente a los requerimientos de comprobantes desarrollados por otro grupo.

Support: correspondiente a los requerimientos de soporte desarrollados por otro grupo.

Esto permite que cada grupo trabaje sobre una parte específica sin perder la estructura general del módulo.

4. Uso de mocks en esta primera etapa

En esta primera etapa, M8 todavía no se conectará directamente con los demás módulos del sistema.

Por eso, la información que más adelante llegará desde módulos como M1, M5, M6 o M7 se reemplazará temporalmente por datos simulados o mocks.

Por ejemplo:

- Para RF-8.1 Notificaciones, se simularán datos como el identificador del viaje, el usuario y el evento ocurrido.
- Para RF-8.2 QR, se simulará principalmente el identificador del viaje al que debe asociarse el código.

Los mocks se utilizarán solamente para reemplazar la información externa que todavía no recibimos de otros módulos.

Las funcionalidades propias de M8 sí deberán funcionar realmente. Es decir, el módulo deberá procesar las notificaciones y generar, validar, controlar el vencimiento y evitar la reutilización del QR.

5. OpenAPI y Swagger UI

La API se documentará utilizando el estándar OpenAPI.

OpenAPI permitirá definir de forma clara el contrato de los endpoints, especificando qué operaciones ofrece la API, qué información recibe cada una, qué respuestas devuelve y qué errores pueden producirse.

El escenario de la cátedra recomienda que las comunicaciones REST/HTTP utilicen contratos bien definidos y documentación OpenAPI.

Para visualizar esa documentación se utilizará Swagger UI.

Swagger UI toma la especificación OpenAPI y la presenta mediante una interfaz web desde la cual se pueden consultar y probar los endpoints.

Por lo tanto, cada tecnología cumple una función diferente:

Express implementa y ejecuta la API.

OpenAPI define y documenta el contrato de la API.

Swagger UI permite visualizar y probar ese contrato.

6. Enfoque para diseñar la API

Existen dos formas habituales de trabajar con OpenAPI.

- **Code-First**, primero se desarrolla la API y después se genera o agrega la documentación OpenAPI sobre lo que ya fue programado.
- **Design-First o Contract-First**, primero se define cómo deberá comportarse la API mediante OpenAPI y luego se desarrolla el código respetando ese contrato.

Nosotros trabajaremos con la forma **Design-First**.

El cual resulta conveniente porque RF-8.1 y RF-8.2 serán desarrollados por dos personas diferentes, pero posteriormente deberán formar parte de la misma API.

Definir previamente el contrato permite acordar nombres, endpoints, datos de entrada, respuestas y errores antes de empezar a programar.

De esta manera, ambos desarrollos parten de una definición común y se reduce la posibilidad de que cada requerimiento termine utilizando formatos incompatibles y además no aparezcan como comentario dentro de la lógica del código lo cual hace que tengamos muchas más líneas de código lo que sería una desventaja.

7. Organización funcional de RF-8.1

La parte de Notificaciones estará organizada dentro de una sección propia de la API.

Se prevé trabajar con componentes destinados a controlar los endpoints, aplicar la lógica de negocio, validar los datos recibidos, utilizar información mock durante esta etapa y realizar las pruebas correspondientes.

RF 8.1 deberá contemplar los eventos definidos por la cátedra:

Solicitud de viaje, asignación, arribo del conductor, inició, cancelación y finalización.

La lógica de M8 será responsable de transformar esos acontecimientos en las notificaciones correspondientes.

8. Organización funcional de RF-8.2

La parte de QR también tendrá una sección propia dentro de la misma API.

Se deberá implementar la generación del QR, su asociación con un viaje, su vencimiento, validación y control de un solo uso.

Además, el QR no deberá exponer información sensible, tal como establece el requerimiento RF 8.2.

La funcionalidad deberá permitir comprobar posteriormente si un QR existe, continúa vigente y no fue utilizado anteriormente.

9. Estructura general prevista

Nuestra parte del Módulo 8 quedará organizada de la siguiente manera:

M8

Notifications — RF-8.1

- controller
- service
- dto
- mock-data
- tests
- documentación OpenAPI

QR — RF-8.2

- controller
- service
- dto
- generación de QR
- validación
- expiración
- tests
- documentación OpenAPI

Además existirán elementos compartidos por la API, como la configuración general de Express, OpenAPI, Swagger UI, Docker, README y las configuraciones necesarias para ejecutar el servicio.

10. Estrategia general de trabajo con Git

El repositorio es compartido por todo el curso, por lo que nuestro trabajo deberá mantenerse separado del desarrollo de los demás módulos.

RF-8.1 y RF-8.2 se desarrollarán inicialmente en branches diferentes:

feature/m8-rf81-notifications

feature/m8-rf82-qr

Posteriormente ambos desarrollos se integrarán en una rama común del grupo:

feature/m8-grupo6

Esta rama contendrá el resultado conjunto de RF-8.1 y RF-8.2 dentro de una misma API de M8.

Durante esta etapa no se trabajará ni se realizarán commits directamente sobre **main**, y tampoco se modificarán branches correspondientes a otros módulos o grupos.

Decisión técnica resumida

La solución de nuestro grupo será una API REST desarrollada con Node.js y Express, utilizando mocks para reemplazar temporalmente las dependencias con otros módulos.

La API será especificada mediante OpenAPI siguiendo un enfoque Design-First, y Swagger UI será utilizado para visualizar y probar los endpoints.

RF-8.1 y RF-8.2 se desarrollarán inicialmente de forma separada, pero ambos deberán formar parte de una única API del Módulo 8.